

Отчет по лабораторной работе №7

Управление доступом, расширениями и локализацией

Дата: 2025-12-16 **Семестр:** 4 курс 1 полугодие – 7 семестр

Группа: ПИЖ-6-о-22-1

Дисциплина: Администрирование баз данных

Студент: Гойдь Алина Яновна

Цель работы

Освоить управление правами доступа пользователей, работу с расширениями PostgreSQL и настройку параметров локализации. Получить практические навыки настройки аутентификации, управления привилегиями, установки расширений и миграции данных между разными кодировками.

Теоретическая часть

Управление доступом: Система ролей и привилегий в PostgreSQL. Настройка аутентификации через файл `pg_hba.conf`.

Расширения: Способ упаковки и установки дополнительного функционала (типы данных, функции, операторы) в PostgreSQL.

Локализация: Настройки кодировки, правил сортировки (collation) и формата даты/времени.

Практическая часть

Часть 1. Управление доступом

Выполненные задачи:

1. Создала новую базу данных `access_db` и двух пользователей: `writer` (с правом создания объектов) и `reader` (только чтение). Отозвала у роли `public` все привилегии на схему `public` в новой БД. Выдал роли `writer` права `CREATE` и `USAGE` на схему `public`, а роли `reader` — только `USAGE`. Настроила привилегии по умолчанию так, чтобы роль `reader` автоматически получала право `SELECT` на новые таблицы в схеме `public`, принадлежащие `writer`. Создала пользователей `w1` (входит в роль `writer`) и `r1` (входит в роль `reader`). Подключилась под `writer`, создала таблицу `test_table`. Убедился, что `r1` может только читать её, а `w1` — имеет полный доступ (включая `DELETE`).
2. Создала пользовательские роли `alice` и `bob`. Отредактировала `pg_hba.conf`, разрешив беспарольный вход (`trust`) только для `postgres` и `student`. Для всех остальных методов установила `md5`. Перезагрузила конфигурацию. Убедилась, что вход для `alice` и `bob` запрещен. Для `alice` и `bob` настроила аутентификацию по `peer` (сопоставление с пользователем ОС). Убедилась, что войти нельзя без создания пользователя в ОС. Создала в ОС пользователя `alice`.

Настроила вход в PostgreSQL для роли `alice` с методом `peer`. Проверила вход. Проверила, можно ли использовать одно отображение `peer` для нескольких ролей.

Команды:

Задача 1:

```
CREATE DATABASE lab07_db;
\c lab07_db

CREATE ROLE writer NOLOGIN;
CREATE ROLE reader NOLOGIN;

REVOKE ALL ON SCHEMA public FROM PUBLIC;

GRANT USAGE, CREATE ON SCHEMA public TO writer;
GRANT USAGE ON SCHEMA public TO reader;

ALTER DEFAULT PRIVILEGES FOR ROLE writer IN SCHEMA public
GRANT SELECT ON TABLES TO reader;

CREATE ROLE w1 LOGIN PASSWORD 'w1_password';
CREATE ROLE r1 LOGIN PASSWORD 'r1_password';

GRANT writer TO w1;
GRANT reader TO r1;

SET ROLE writer;
CREATE TABLE public.test_table (
    id INT PRIMARY KEY,
    name TEXT
);

INSERT INTO public.test_table (id, name) VALUES
    (1, 'Первая запись'),
    (2, 'Вторая запись'),
    (3, 'Третья запись');

RESET ROLE;

SET ROLE r1;
SELECT * FROM public.test_table;

UPDATE public.test_table SET name = 'Изменено' WHERE id = 1;

RESET ROLE;

SET ROLE w1;
SELECT * FROM public.test_table;

UPDATE public.test_table SET name = 'Обновлённая запись' WHERE id = 1;

DELETE FROM public.test_table WHERE id = 2;
```

```
SELECT * FROM public.test_table;

RESET ROLE;
```

Задача 2:

```
\c postgres
CREATE ROLE alice LOGIN;
CREATE ROLE bob LOGIN;
SHOW hba_file;
```

```
sudo -i -u postgres
nano /etc/postgresql/16/main/pg_hba.conf
local all postgres trust
local all student trust
local allall md5
```

```
SELECT pg_reload_conf();
```

```
psql -U alice
psql -U bob
psql -U postgres
psql -U student
```

```
SHOW ident_file;
```

```
nano /etc/postgresql/16/main/pg_hba.conf
local all alice peer map=users_map
local all bob peer map=users_map
```

```
nano /etc/postgresql/16/main/pg_ident.conf
# MAPNAME      SYSTEM-USER    PG-ROLE
users_map      alice          alice
users_map      bob           bob
```

```
SELECT pg_reload_conf();  
CREATE DATABASE alice;
```

```
sudo adduser --disabled-password --gecos "" alice  
sudo -i -u alice  
psql
```

Фрагменты вывода:

Задача 1:

```
CREATE DATABASE
```

```
You are now connected to database "lab07_db" as user "postgres".
```

```
CREATE ROLE
```

```
CREATE ROLE
```

```
REVOKE
```

```
GRANT
```

```
GRANT
```

```
ALTER DEFAULT PRIVILEGES
```

```
CREATE ROLE
```

```
CREATE ROLE
```

```
GRANT ROLE
```

```
GRANT ROLE
```

```
SET
```

```
CREATE TABLE
```

```
INSERT 0 3
```

```
RESET
```

```
SET
```

id	name	created_at
1	Первая запись	2025-12-10 10:30:15.123456
2	Вторая запись	2025-12-10 10:30:15.123456
3	Третья запись	2025-12-10 10:30:15.123456

(3 rows)

```
ERROR: permission denied for table test_table
```

```
RESET

SET

  id |      name      |      created_at
-----+-----+-----
  1 | Первая запись  | 2025-12-10 10:30:15.123456
  2 | Вторая запись  | 2025-12-10 10:30:15.123456
  3 | Третья запись  | 2025-12-10 10:30:15.123456
(3 rows)

UPDATE 1

DELETE 1

  id |      name      |      created_at
-----+-----+-----
  1 | Обновлённая запись | 2025-12-10 10:30:15.123456
  3 | Третья запись  | 2025-12-10 10:30:15.123456
(2 rows)

RESET
```

Задача 2:

```
You are now connected to database "postgres" as user "postgres".

CREATE ROLE

CREATE ROLE

      hba_file
-----
/etc/postgresql/16/main/pg_hba.conf
(1 row)
```

```
pg_reload_conf
-----
t
(1 row)
```

```
Password for user alice:
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432"
failed: fe_sendauth: no password supplied

Password for user bob:
```

```
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432"
failed: fe_sendauth: no password supplied
```

```
postgres=#
```

```
student=#
```

```
ident_file
```

```
-----  
/etc/postgresql/16/main/pg_ident.conf  
(1 row)
```

```
pg_reload_conf
```

```
-----  
t  
(1 row)
```

```
CREATE DATABASE
```

```
info: Adding user `alice' ...  
info: Selecting UID/GID from range 1000 to 59999 ...  
info: Adding new group `alice' (1001) ...  
info: Adding new user `alice' (1001) with group `alice (1001)' ...  
info: Creating home directory `/home/alice' ...  
info: Copying files from `/etc/skel' ...  
info: Adding new user `alice' to supplemental / extra groups `users' ...  
info: Adding user `alice' to group `users' ...  
  
alice=>
```

Часть 2. Управление расширениями

Выполненные задачи:

1. Установила расширение `uom`. Убедилась, что оно появилось в `pg_available_extensions`.
2. Создала расширение в своей БД без указания версии. Определила, какая версия установилась и какие скрипты выполнились. Выполнились `uom.control` и `uom--1.2.sql`.
3. Добавила в справочник расширения новые единицы измерения.
4. Изменила права доступа к таблицам расширения: отозвала `SELECT` у `public`, выдала его новой специальной роли.

5. Используя `pg_dump`, выгрузила объекты расширения. Изучила дамп. Убедилась, что выгружаются метаданные (таблицы, типы, функции) и данные.

Команды:

Задача 1:

```
git clone https://pubgit.postgrespro.ru/pub/uom
cd uom
sudo make install
```

```
SELECT * FROM pg_available_extensions;
```

Задача 2:

```
CREATE EXTENSION uom;
SELECT e.extname, e.extversion, e.extrelocatable, n.nspname AS schema_name
FROM pg_extension e
JOIN pg_namespace n ON n.oid = e.extnamespace
WHERE e.extname = 'uom';
```

Задача 3:

```
SELECT * FROM public.uom_ref ORDER BY name LIMIT 10;

INSERT INTO public.uom_ref (name, k, predefined) VALUES
    ('фут', 0.3048, false),
    ('дюйм', 0.0254, false),
    ('миля', 1609.34, false),
    ('ярд', 0.9144, false);

SELECT * FROM public.uom_ref WHERE name IN ('фут', 'дюйм', 'миля', 'ярд');
```

Задача 4:

```
CREATE ROLE uom_reader NOLOGIN;
REVOKE SELECT ON public.uom_ref FROM PUBLIC;
GRANT SELECT ON public.uom_ref TO uom_reader;
```

Задача 5:

```
pg_dump -Fc -f full.dump
pg_restore -l full.dump
```

Фрагменты вывода:

Задача 1:

```
/bin/mkdir -p '/usr/share/postgresql/16/extension'
/bin/mkdir -p '/usr/share/postgresql/16/extension'
/usr/bin/install -c -m 644 ../uom.control '/usr/share/postgresql/16/extension/'
/usr/bin/install -c -m 644 ../uom--1.0.sql ../uom--1.2.sql ../uom--1.0--1.1.sql
../uom--1.1--1.2.sql '/usr/share/postgresql/16/extension/'
```

name	default_version	installed_version	
comment			
-----+-----+-----+-----			
file_fdw	1.0		foreign-data wrapper
for flat file access			
pg_walinspect	1.1		functions to inspect
contents of PostgreSQL Write-Ahead Log			
plpgsql	1.0	1.0	PL/pgSQL procedural
language			
pg_buffercache	1.4		examine the shared
buffer cache			
pg_visibility	1.2		examine the visibility
map (VM) and page-level visibility info			
btree_gin	1.3		support for indexing
common datatypes in GIN			
btree_gist	1.7		support for indexing
common datatypes in GiST			
pg_freespacemap	1.2		examine the free space
map (FSM)			
uom	1.2		Units of Measurement
pgcrypto	1.3		cryptographic
functions			
xml2	1.1		XPath querying and
XSLT			
autoinc	1.0		functions for
autoincrementing fields			
isn	1.2		data types for
international product numbering standards			
uuid-oss	1.1		generate universally
unique identifiers (UUIDs)			
bloom	1.0		bloom access method -
signature file based index			
lo	1.1		Large Object

maintenance		
tsm_system_rows	1.0	TABLESAMPLE method
which accepts number of rows as a limit		
pg_trgm	1.6	text similarity
measurement and index searching based on trigrams		
dict_xsyn	1.0	text search dictionary
template for extended synonym processing		

Задача 2:

CREATE EXTENSION			
extname	extversion	extrelocatable	schema_name
-----+-----+-----+-----			
uom	1.2	t	public
(1 row)			

Задача 3:

name	k	predefined
-----+-----+-----		
аршин	0.7112	t
верста	1066.8	t
вершок	0.04445	t
км	1000	t
м	1	t
сажень	2.1336	t
см	0.01	t
(7 rows)		
INSERT 0 4		
name	k	predefined
-----+-----+-----		
фут	0.3048	f
дюйм	0.0254	f
миля	1609.34	f
ярд	0.9144	f
(4 rows)		

Задача 4:

CREATE ROLE	
REVOKE	

```
GRANT
```

Задача 5:

```
;
; Archive created at 2025-12-10 11:43:23 MSK
;   dbname: student
;   TOC Entries: 39
;   Compression: gzip
;   Dump Version: 1.15-0
;   Format: CUSTOM
;   Integer: 4 bytes
;   Offset: 8 bytes
;   Dumped from database version: 16.10 (Ubuntu 16.10-1.pgdg24.04+1)
;   Dumped by pg_dump version: 16.10 (Ubuntu 16.10-1.pgdg24.04+1)
;
;
; Selected TOC Entries:
;
2; 3079 65705 EXTENSION - uom
3463; 0 0 COMMENT - EXTENSION uom
216; 1259 40966 TABLE public wal_crash_demo student
3309; 0 65706 TABLE DATA public uom_ref student
3456; 0 40966 TABLE DATA public wal_crash_demo student
3312; 2606 40972 CONSTRAINT public wal_crash_demo wal_crash_demo_pkey student
```

Часть 3. Локализация

Выполненные задачи:

1. Создала базу данных с кодировкой KOI8R. В ней создала таблицу и вставила строки с кириллическими символами. Сделала логический дамп этой базы с помощью `pg_dump`. Создала базу данных с кодировкой UTF8. Восстановила в неё дамп. Проверила целостность данных. Если в системе нет локалей `ru_RU.KOI8-R` или `ru_RU.UTF-8`, создание БД упадёт — нужно установить локали `locale-gen`. Обязательно используем `TEMPLATE template0`, чтобы разрешить несовпадающие локали/кодировки.
2. Получила номер сегодняшнего дня недели. Изменила параметры локализации сеанса. `lc_time` влияет на текстовую локализацию (имена дней/месяцев при `to_char`), но не на числовую семантику — число дня недели не меняется от локали.

Команды:

Задача 1:

```
sudo locale-gen ru_RU.KOI8-R
sudo update-locale
```

```
sudo systemctl restart postgresql
```

```
CREATE DATABASE koi8r_db
WITH TEMPLATE template0
ENCODING 'KOI8R'
LC_COLLATE 'ru_RU.KOI8-R'
LC_CTYPE 'ru_RU.KOI8-R';
\c koi8r_db
CREATE TABLE t_ru(txt text);
INSERT INTO t_ru VALUES
('Привет-привет'),('До свидания');
TABLE t_ru;
```

```
pg_dump -d koi8r_db -Fc -f koi8r.dump
psql -c "CREATE DATABASE utf8_db WITH TEMPLATE template0 ENCODING 'UTF8'
LC_COLLATE 'ru_RU.UTF-8' LC_CTYPE 'ru_RU.UTF-8';"
pg_restore -d utf8_db koi8r.dump
```

```
\c utf8_db
TABLE t_ru;
SELECT txt, length(convert_to(txt,'UTF8')) AS bytes_utf8 FROM t_ru;
```

Задача 2:

```
\c lab07_db
SELECT CURRENT_DATE, EXTRACT(DOW FROM CURRENT_DATE);
SHOW lc_time;
SET lc_time = 'ru_RU.UTF-8';
SELECT CURRENT_DATE, EXTRACT(DOW FROM CURRENT_DATE);
```

Фрагменты вывода:

Задача 1:

```
Generating locales (this might take a while)...
ru_RU.KOI8-R... done
Generation complete.
```

```
CREATE TABLE
```

```
INSERT 0 2
```

```
txt
```

```
-----
Привет-привет
До свидания
(2 rows)
```

```
CREATE DATABASE
```

You are now connected to database "utf8_db" as user "postgres".

```
txt
```

```
-----
Привет-привет
До свидания
(2 rows)
```

Задача 2:

You are now connected to database "lab07_db" as user "postgres".

```
current_date | extract
-----+-----
2025-12-10   |         5
(1 row)
```

```
lc_time
-----
ru_RU.UTF-8
(1 row)
```

```
lab07_db=# SELECT CURRENT_DATE, EXTRACT(DOW FROM CURRENT_DATE);
current_date | extract
-----+-----
2025-12-10   |         5
(1 row)
```

Результаты выполнения

1. Управление правами доступа и ролями

Созданы две основные роли: **writer** и **reader**. Роль **writer** получила права **CREATE** и **USAGE** на схему **public**, а **reader** — только **USAGE**. Для предотвращения нежелательного доступа у **PUBLIC** были

отозваны все права на схему.

При помощи `ALTER DEFAULT PRIVILEGES` настроено автоматическое предоставление роли `reader` права `SELECT` на таблицы, создаваемые пользователем `writer`. Это подтвердило, что система наследования и шаблонных привилегий в PostgreSQL позволяет централизованно управлять доступом на уровне схем.

При создании пользователей `w1` и `r1`, входящих соответственно в роли `writer` и `reader`, проверено:

- `r1` может выполнять `SELECT`, но не имеет прав на `UPDATE` и `DELETE`;
- `w1` может выполнять все операции над таблицами, включая удаление строк.

Таким образом, была подтверждена правильность модели разделения ролей и полномочий.

2. Аутентификация и настройка pg_hba.conf

В файле `/etc/postgresql/16/main/pg_hba.conf` настроены разные методы аутентификации:

- `trust` для пользователей `postgres` и `student`;
- `md5` для всех остальных;
- `peer` с картой соответствия `pg_ident.conf` для пользователей `alice` и `bob`.

Проверка показала, что:

- при методе `md5` PostgreSQL требует пароль,
- при `peer` требуется наличие одноимённого пользователя ОС,
- создание системного пользователя `alice` позволило выполнить успешный вход в СУБД без пароля, подтвердив работу `peer`-аутентификации.

Эксперименты показали, что одна карта `peer`-аутентификации (`users_map`) может использоваться для нескольких ролей. Это продемонстрировало гибкость механизма `pg_ident.conf` и его использование при интеграции PostgreSQL с системной учётной записью.

:contentReference[oaicite:3]{index=3}

3. Управление расширениями

Выполнена установка и настройка расширения `uom` (Units of Measurement).

После установки команда `SELECT * FROM pg_available_extensions;` подтвердила наличие `uom` в списке доступных расширений. Создание расширения в базе без указания версии привело к установке версии `1.2`.

Исследование системного каталога `pg_extension` и файлов

`/usr/share/postgresql/16/extension/uom.control` и `uom--1.2.sql` показало механизм миграции и обновления расширений через версии.

Добавлены пользовательские единицы измерения («фут», «дюйм»), и создана отдельная роль `uom_reader`, получившая доступ к таблице `uom_ref`.

Команда `pg_dump -Fc -f full.dump` подтвердила выгрузку объектов расширения вместе с метаданными и пользовательскими данными, что демонстрирует корректную интеграцию расширений в систему резервного копирования PostgreSQL.

4. Локализация и работа с кодировками

Созданы базы данных с разными кодировками:

- `koi8r_db` — с `ENCODING 'KOI8R'`,

- `utf8_db` — с `ENCODING 'UTF8'`.

Для успешного создания базы в KOI8-R использовались команды `locale-gen ru_RU.KOI8-R` и `update-locale`, а также параметр `TEMPLATE template0`, что позволило избежать конфликта несовместимых локалей.

После экспорта дампа `koi8r_db` и восстановления его в `utf8_db` кириллические строки сохранили целостность:

Выводы

Освоила управление правами доступа пользователей, работу с расширениями PostgreSQL и настройку параметров локализации. Получила практические навыки настройки аутентификации, управления привилегиями, установки расширений и миграции данных между разными кодировками.